Streaming Raking in Fixed Memory*

Gaurav Sood†

October 12, 2026

Abstract

Raking adjusts survey weights so that weighted shares of demographic groups match population targets. The standard algorithm, iterative proportional fitting (IPF), needs every row seen so far, so keeping weights current on a stream means storing the stream and refitting as it grows. We give a raking method whose state is a fixed set of running totals. Each arriving row receives the weight that minimizes its raking distance from its design weight plus a penalty on the running gap between weighted and target margins; earlier weights stay fixed. This one-dimensional problem has a closed-form solution through the Wright omega function, so each arrival costs $O(p)$ time for p margins and the method keeps $p + 4$ numbers no matter how long the stream runs. The same objective minimized over all rows at once is entropy balancing with a ridge penalty, which tends to IPF as the penalty grows. In simulations across six scenarios (50 replications each), the error of a weighted outcome mean at the end of the stream is no more than 27% above full-data IPF when targets are moderate, and 2% below it in one scenario, while storing 8 to 12 numbers instead of 2,500 to 450,000. During the stream, it tracks the margins more closely than IPF refit every 1,000 rows or at each doubling of the stream in five of six scenarios, usually at a fraction of the compute. It does worse with extreme targets and with 50 margins (bias 11% and 52% above IPF), and its estimates depend on arrival order. When the rows can be stored, periodic IPF remains the better choice.

1 Introduction

Survey weights are often adjusted so that the weighted sample matches known population shares: the weighted share of women should equal the census share of women, and likewise for age groups, regions, and education levels. When only the marginal shares are known, the standard adjustment is *raking*, computed by iterative proportional fitting (IPF; Deming and Stephan, 1940). IPF takes each characteristic in turn and rescales the weights so that characteristic's weighted share matches its target, and cycles until all shares match.

*Code, data, and replication scripts: https://github.com/finite-sample/onlinerake. The method is `StreamingRaker` in the `onlinerake` Python package.

†Gaurav can be reached at gsood07@gmail.com

IPF needs every row each time it runs. That is no problem for a finished survey, but many samples arrive as streams: online panels, rolling surveys, app telemetry, river samples. Weighted estimates are wanted while data arrive, and keeping weights current means storing every row and refitting as the sample grows. Refitting is affordable at survey scale: one full IPF fit over 50,000 rows and 8 margins took about 0.3 seconds in our runs. The binding constraints are elsewhere. Sometimes rows cannot be kept, because they contain personal data that should not be retained or because the volume is too large. Sometimes weighting happens on a device or in a pipeline with a fixed memory budget. And the cost of refitting grows with the stream: in our measurements, refitting every 1,000 arrivals costs about 42 microseconds per arrival at 10,000 rows and 5,685 at a million (Section 4).

This paper asks what raking is possible when all one can keep is a fixed set of running totals, and what that costs in accuracy. Our answer has three parts. First, we write raking as a penalized problem (Section 3.1): weights should stay close to their design weights in the raking distance, and the running totals of weighted deviations from the targets should be small. Second, holding earlier weights fixed, the best weight for an arriving row has a closed form (Section 3.2). The update needs only the running totals, costs $O(p)$, and has one tuning parameter, the penalty strength, which says how much weight variability to accept to close the gap to the targets. Third, the same objective over all rows has an exact solution through a p-dimensional dual (Section 3.3), which isolates the cost of not storing the data.

The experiments (Section 4) compare the streaming method with full-data IPF, with IPF refit periodically on the stored stream, and with online methods that store every row and re-optimize all weights at each arrival.

2 Background

2.1 Raking as calibration

Let row i have a vector $x_i \in \{0,1\}^p$ of group indicators (for example $x_{i1} = 1$ for women and $x_{i2} = 1$ for people under 30) and a design weight $d_i > 0$, which is one when there is no sampling design. Let $t \in (0,1)^p$ be the target shares. The weighted share of group j under weights w is

$$m_j(w) = \frac{\sum_i w_i x_{ij}}{\sum_i w_i}. \tag{1}$$

Usually many weight vectors give $m(w) = t$. Calibration (Deville and Särndal, 1992) picks the one closest to the design weights, where "closest" is set by a distance function. With the distance

$$D(w,d) = w\log(w/d) - w + d, \tag{2}$$

summed over rows, calibration on group margins is raking (Deville and Särndal, 1992); IPF computes it (Csiszár, 1975). Other distances give other estimators; the chi-square distance gives the generalized regression estimator. Entropy balancing (Hainmueller, 2012) solves the raking problem through its dual: the optimal weights are an exponential tilt of the design weights, $w_i = d_i \exp(-\lambda \cdot (x_i - t))$, and the p multipliers λ are found by convex optimization.

2.2 Penalizing instead of constraining

Exact constraints are not the only option. Bardsley and Chambers (1984) proposed replacing the requirement that weighted totals equal their targets with a quadratic penalty on the difference, which trades a little bias for less variable weights. That idea is what makes a streaming version possible, because a penalty, unlike a constraint, can be partly satisfied by each arriving row.

3 Streaming penalized raking

3.1 Objective

Write $\delta_i = x_i - t$ for how far row i sits from the targets, and

$$r(w) = \sum_i w_i \delta_i$$

for the running totals of weighted deviations. Since $r(w) = W\,(m(w) - t)$ with $W = \sum_i w_i$, the margins equal the targets exactly when $r = 0$. We minimize

$$F(w) = \sum_i D(w_i, d_i) + \frac{c}{2}\,\|r(w)\|^2. \tag{3}$$

The first term keeps weights near their design weights; the second pulls the margins toward the targets, with calibration strength $c > 0$ setting the exchange rate. F is strictly convex. As c grows, its minimizer approaches the raking solution whenever the targets are attainable (Section 3.3).

3.2 The arrival update

When row i arrives, all earlier weights are fixed and summarized by the running totals r_0 accumulated so far. The only choice left is w_i, which should minimize

$$f(w) = D(w, d_i) + \frac{c}{2}\|r_0 + w\,\delta_i\|^2.$$

Its derivative is

$$f'(w) = \log(w/d_i) + c\,\delta_i \cdot r_0 + c\,q\,w, \qquad q = \|\delta_i\|^2.$$

Setting $f'(w) = 0$ and substituting $u = c\,q\,w$ gives

$$u + \log u = \log(c\,q\,d_i) - c\,\delta_i \cdot r_0.$$

The solution of $u + \log u = y$ is the Wright omega function $\omega(y)$ (Corless and Jeffrey, 2002), a standard special function available in SciPy. So

$$w_i = \frac{\omega(\log(c\,q\,d_i) - c\,\delta_i \cdot r_0)}{c\,q}, \tag{4}$$

and $w_i = d_i$ when $q = 0$, that is, when the row sits exactly on the targets. Because f is convex, clipping w_i to bounds $[\varepsilon, M]$ gives the bounded minimizer; we use $[0.001, 100]$. The row then joins the running totals: $r \leftarrow r_0 + w_i\delta_i$.

Solving for w_i exactly matters. A simpler rule, a fixed-size gradient step on the arriving weight against the squared margin error, does not calibrate. After k rows that gradient is $(1/W)\sum_j(m_j - t_j)(x_{ij} - m_j)$, where the margin error is of order $1/k$ and the total weight W is of order k, so it shrinks like $1/k^2$: a fixed step leaves each new weight near its design weight and the margins near their unweighted values. The closed form has no step size to tune.

The update reads naturally. If the row's deviation points against the current gap ($\delta_i \cdot r_0 < 0$, for example a man arriving when men are under-represented), the argument of ω is large and the row is weighted up; if it points with the gap, it is weighted down. The strength c sets how hard each row pushes. The optimality condition can also be written $w_i = d_i \exp(-c\, r \cdot \delta_i)$ with r the totals after the update: the same exponential tilt as entropy balancing, with the running quantity $c\, r$ in place of the multipliers.

Cost and state. Each arrival needs two inner products of length p and one evaluation of ω: $O(p)$ time, independent of how many rows came before. The state is r (p numbers), the total weight, the sum of squared weights (for the effective sample size), and the count. Margins are $t + r/W$ at any moment. To estimate the mean of an outcome y, the state also keeps $\sum_i w_i y_i$; the estimate is that sum divided by W. With one outcome, the state is $p + 4$ numbers and no rows are stored.

3.3 The batch counterpart

With all rows available, the minimizer of (3) satisfies $w_i = d_i \exp(-\lambda \cdot \delta_i)$ with $\lambda = c\, r(w)$, so it is found from the p-dimensional convex dual

$$G(\lambda) = \sum_i d_i \exp(-\lambda \cdot \delta_i) + \frac{\|\lambda\|^2}{2c},$$

which is the entropy balancing dual plus a ridge penalty on the multipliers. We solve it with Newton's method using the analytic gradient and Hessian. As $c \to \infty$ the ridge term vanishes and the solution is the raking solution: at $c = 10^6$ it matches IPF weights to 10^{-6} relative error in the package's tests. Comparing this batch solution with the single pass at the same c isolates what is lost by never revisiting a weight.

4 Experiments

4.1 Setup

Scenarios. Each replication draws binary group indicators independently, row by row. *Standard*: $n = 3{,}000$ rows, $p = 8$ margins, sample shares between 0.3 and 0.7, targets within 0.15 of them. *Large*: the same with $n = 50{,}000$. *Hard targets*: sample shares between 0.4 and 0.6 and targets of 0.08 or 0.92. *Many margins*: as Standard with $p = 50$. *Drift*: $n = 5{,}000$,

with sample shares moving linearly to their complements over the stream and targets of 0.5. *Small*: $n = 500$, $p = 4$. There are 50 replications per scenario.

Outcome and truth. Each row has an outcome $y = 1+\beta\cdot x+x_1x_2+\epsilon$ with $\beta_j \sim U(-1,1)$ and $\epsilon \sim N(0,1)$. In the population, groups are independent with shares t, so the true mean is $1+\beta\cdot t+t_1t_2$. The interaction term matters: a linear outcome would be estimated without bias by any weights that hit the margins, while the interaction rewards weights that also keep the groups independent, as IPF's product-form weights do.

Methods. *IPF*: full-data raking at the end of the stream. *Stream*: the update of Section 3.2 with $c \in \{0.01, 0.03, 0.1, 0.3, 1\}$; $c = 0.3$ is the package default and the one reported in the tables. *Batch*: the minimizer of (3) at the same c. *Periodic IPF*: IPF refit on all stored rows every 100 or 1,000 rows, or whenever the stream doubles in length; between refits, a new row gets the product of the last fit's per-group factors, which is the weight IPF would have given it, and the first refits come at 1, 2, 4, ... rows so the warm-up period is covered. *All-weights SGD* and *MWU*: online classes, also in the `onlinerake` package, that store every row and re-optimize all weights at each arrival (run only for $n \leq 5{,}000$ because their cost grows with n^2).

Metrics. At the end of the stream: absolute bias of the weighted outcome mean, averaged over replications, and effective sample size ESS $= (\sum w)^2/\sum w^2$ as a fraction of n. During the stream: mean absolute margin error averaged over 20 checkpoints (every 5% of the stream). Cost: wall-clock microseconds per arrival, including every refit for periodic IPF. Memory: numbers retained.

The timings compare a Python per-row loop (the stream) with vectorized NumPy refits (IPF), on one machine whose load varied between runs; absolute times differed by a factor of two to four across runs for every method. We therefore read the timings as ratios within a run.

4.2 End-of-stream accuracy

Table 1. Absolute bias of the weighted outcome mean and effective sample size at the end of the stream. Stream and Batch use $c = 0.3$. Means over 50 replications.

	IPF	Batch	Stream	All-weights SGD	IPF ESS/n	Stream ESS/n
Standard	0.015	0.015	0.017	0.023	0.771	0.676
Large	0.003	0.003	0.004	–	0.775	0.674
Hard targets	0.128	0.116	0.142	0.165	0.014	0.026
Many margins	0.025	0.025	0.038	0.053	0.232	0.270
Drift	0.014	0.014	0.014	0.017	0.998	0.750
Small	0.040	0.039	0.039	0.039	0.886	0.819

Table 1 reports how much accuracy fixed memory costs. With moderate targets, the stream's bias is close to full-data IPF: 0.017 against 0.015 in Standard, 0.014 against 0.014 under Drift,

0.039 against 0.040 in Small, and 0.004 against 0.003 in Large, where both are small. Its effective sample size is lower, 0.68 of n against 0.77 in Standard: weights set once, with no later correction, vary more.

Two scenarios show the limits. With extreme targets, the stream's bias is 0.142 against IPF's 0.128, and with 50 margins it is 0.038 against 0.025. The batch solution at the same c matches or beats IPF in both (0.116 and 0.025), so the gap comes from never revisiting weights, not from the penalty. The all-weights SGD class has higher bias than the stream in all five scenarios where it ran, while storing every row.

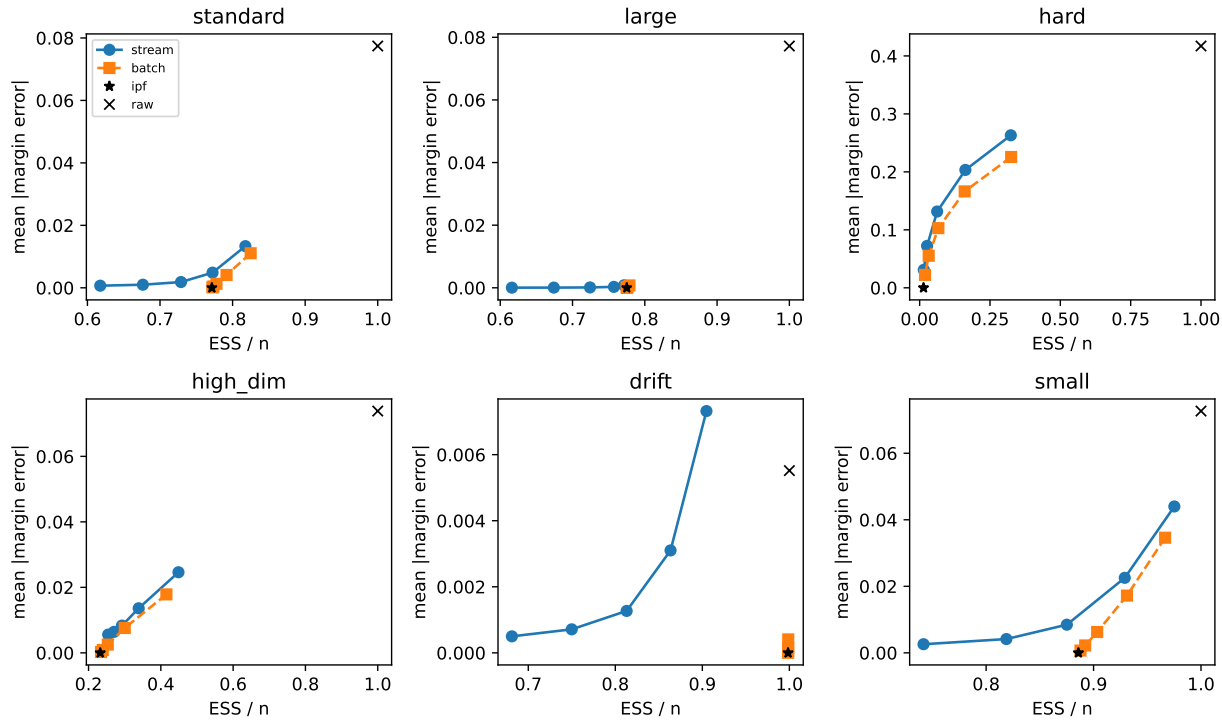


Figure 1. Margin error against effective sample size as the calibration strength c varies from 0.01 to 1. Circles: stream. Squares: batch solution of the same objective. Star: full-data IPF. Cross: unweighted.

Figure 1 shows the choice c makes. Larger c moves the stream toward the targets and lowers its effective sample size. The gap between the stream and batch curves is again the price of a single pass: negligible in Standard, Large, and Drift, visible with extreme targets and many margins.

4.3 During the stream, and what it costs

Table 2. During the stream: mean absolute margin error over 20 checkpoints, and microseconds per arrival. Stream uses $c = 0.3$. In Small ($n = 500$) the 1,000-row cadence never fires. Means over 50 replications.

	Stream	IPF every 1000	IPF at doubling	Stream us	IPF every 1000 us	IPF at doubling us
Standard	0.0035	0.0075	0.0073	25	169	181
Large	0.0002	0.0004	0.0019	20	81	16
Hard targets	0.1263	0.0227	0.0219	13	201	200
Many margins	0.0162	0.0730	0.0724	27	2147	2144
Drift	0.0021	0.0111	0.0192	28	109	117
Small	0.0140	–	0.0241	59	–	831

Table 2 compares the stream with periodic IPF at cadences that cost about the same or more. In five of six scenarios the stream tracks the margins more closely than IPF refit every 1,000 rows or at each doubling: 0.0035 against 0.0075 in Standard and 0.016 against 0.073 with many margins, while costing 13 to 59 microseconds per arrival against 81 to 2,147 for IPF every 1,000 rows. The exception is Hard targets, where IPF tracks far better (0.022 against 0.126). In Large, refitting only at doublings is cheaper than the stream (16 against 20 microseconds), though less accurate. Refitting every 100 rows tracks better than the stream in four of six scenarios, but not with many margins or in Small, and costs 214 to 3,944 microseconds per arrival (Appendix Table 3). Figure 2 shows these comparisons for every c.

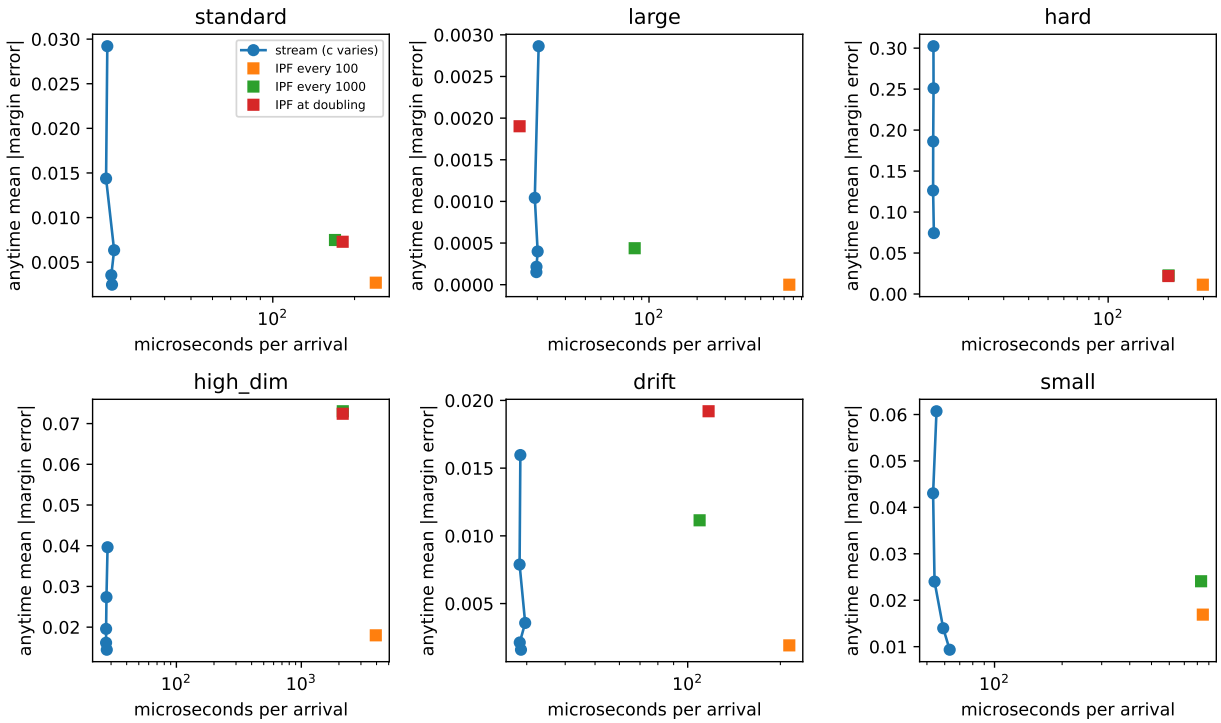


Figure 2. Mean absolute margin error during the stream against microseconds per arrival. Circles: stream as c varies. Squares: periodic IPF.

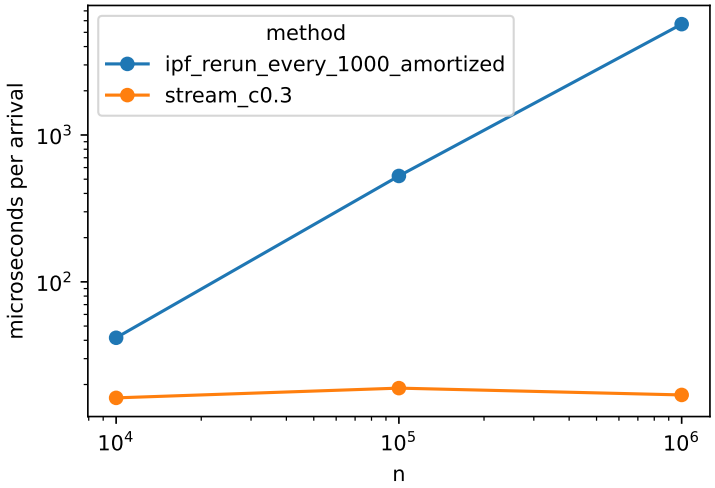


Figure 3. Microseconds per arrival as the stream grows: the stream against IPF refit every 1,000 arrivals (one full refit's time divided by 1,000).

Figure 3 shows the reason for streaming. The stream's cost per arrival does not change with the length of the stream: 16, 19, and 17 microseconds at 10,000, 100,000, and 1,000,000 rows. Refitting IPF every 1,000 arrivals costs 42, 526, and 5,685 microseconds per arrival at the same sizes, and requires storing every row: $n(p+1)$ numbers, or 27,000 in Standard and 450,000 in Large, against 12 for the stream.

4.4 Arrival order

Because earlier weights are never revisited, the stream's estimate depends on the order in which rows arrive. Reshuffling the same 3,000 rows ten times in each of 20 Standard replications, the estimate of the outcome mean varies with a standard deviation of 0.0086. The sampling variation of the same estimator across replications is 0.020. Order therefore adds noise equal to about two-fifths of the sampling noise, which matters for reporting uncertainty: a confidence interval that ignores it is too narrow.

5 Discussion

When to use what. If the rows can be stored and the stream is of survey size, refit IPF periodically: it is exact at each refit, has the highest effective sample size, and its cost is small at these sizes. Use the streaming method when the rows cannot or should not be stored, when memory is fixed, or when the stream is long enough that refitting dominates the cost. Choose c by Figure 1: around 0.3 for moderate targets, larger when targets are far from the sample.

Why a single pass works as well as it does. Each arrival sees the current gap r and leans against it, so under-represented groups are pulled up by the rows that arrive later. Early weights are set when the gap is noisy and are never corrected, which is why the effective

sample size falls and why order matters. With many margins, each arrival can correct only one direction of a high-dimensional gap, and the single pass falls further behind the batch solution.

Limitations. We have no convergence guarantee for the single pass; the evidence is the simulations. The scenarios draw groups independently; correlated groups, continuous calibration variables (which the package supports as mean targets), and adversarial orderings are untested. Drift is handled only in the sense that the weights follow the stream; there is no forgetting, so a stream whose population changes will be weighted toward its average. Uncertainty estimates for the streaming estimator, including the order component, remain to be developed.

6 Conclusion

Raking needs the data, but the streaming version needs only $p + 4$ running totals. Each arriving row receives the weight that best trades closeness to its design weight against the running gap to the targets, in closed form. In simulations it comes close to full-data IPF when targets are moderate, tracks the margins during the stream better than IPF refit at similar cost, and keeps a constant cost per arrival as the stream grows. It falls short with extreme targets and many margins, and its estimates depend on arrival order. Where rows can be stored, periodic IPF remains the better tool.

References

Bardsley, P. and Chambers, R. L. (1984). Multipurpose estimation from unbalanced samples. *Applied Statistics*, 33(3):290.

Corless, R. M. and Jeffrey, D. J. (2002). The Wright ω function. In *Artificial Intelligence, Automated Reasoning, and Symbolic Computation*, volume 2385 of *Lecture Notes in Computer Science*, pages 76–89. Springer.

Csiszár, I. (1975). I-divergence geometry of probability distributions and minimization problems. *The Annals of Probability*, 3(1):146–158.

Deming, W. E. and Stephan, F. F. (1940). On a least squares adjustment of a sampled frequency table when the expected marginal totals are known. *Annals of Mathematical Statistics*, 11(4):427–444.

Deville, J.-C. and Särndal, C.-E. (1992). Calibration estimators in survey sampling. *Journal of the American Statistical Association*, 87(418):376–382.

Hainmueller, J. (2012). Entropy balancing for causal effects: A multivariate reweighting method to produce balanced samples in observational studies. *Political Analysis*, 20(1):25–46.

A All methods and scenarios

Table 3. All methods and scenarios, means over 50 replications. Columns: final mean absolute margin error, mean absolute margin error during the stream, ESS as a fraction of n, absolute bias of the outcome mean at the end and during the stream, and microseconds per arrival.

scenario	method	final_err	anytime_err	ess_frac	abs_bias	anytime_abs_bias	us_per_obs
standard	raw	0.0774	0.0777	1.0000	0.1187	0.1226	0.0000
	ipf	0.0000	0.0000	0.7714	0.0152	0.0274	6.9147
	periodic_ipf_100	0.0000	0.0027	0.7714	0.0152	0.0277	239.3956
	periodic_ipf_1000	0.0000	0.0075	0.7714	0.0152	0.0312	169.4148
	periodic_ipf_doubling	0.0000	0.0073	0.7714	0.0152	0.0315	180.7990
	batch_c0.3	0.0004	–	0.7736	0.0153	–	1.1723
	stream_c0.3	0.0010	0.0035	0.6764	0.0165	0.0303	25.4522
	stream_c1	0.0007	0.0025	0.6176	0.0169	0.0308	25.6089
	sgd_all	0.0093	–	0.7217	0.0234	–	752.4061
	mwu_all	0.0311	–	0.6926	0.0526	–	901.3959
large	raw	0.0772	0.0774	1.0000	0.1164	0.1170	0.0000
	ipf	0.0000	0.0000	0.7750	0.0033	0.0062	5.8602
	periodic_ipf_100	0.0000	0.0000	0.7750	0.0033	0.0062	754.5289
	periodic_ipf_1000	0.0000	0.0004	0.7750	0.0033	0.0064	81.3624
	periodic_ipf_doubling	0.0000	0.0019	0.7750	0.0033	0.0069	15.5011
	batch_c0.3	0.0000	–	0.7751	0.0033	–	0.5844
	stream_c0.3	0.0001	0.0002	0.6741	0.0042	0.0068	19.7700
	stream_c1	0.0000	0.0002	0.6161	0.0046	0.0072	19.7419
hard	raw	0.4171	0.4174	1.0000	0.5522	0.5519	0.0000
	ipf	0.0000	0.0043	0.0137	0.1281	0.2116	4.4456
	periodic_ipf_100	0.0000	0.0114	0.0137	0.1281	0.2209	297.8143
	periodic_ipf_1000	0.0000	0.0227	0.0137	0.1281	0.2262	200.7258
	periodic_ipf_doubling	0.0000	0.0219	0.0137	0.1281	0.2294	200.2702
	batch_c0.3	0.0552	–	0.0321	0.1163	–	1.0708
	stream_c0.3	0.0723	0.1263	0.0264	0.1425	0.2022	13.2895
	stream_c1	0.0310	0.0743	0.0147	0.1376	0.1761	13.4082
	sgd_all	0.0394	–	0.0107	0.1645	–	400.8079
	mwu_all	0.1674	–	0.0255	0.2406	–	483.9236
high_dim	raw	0.0738	0.0749	1.0000	0.2680	0.2736	0.0000
	ipf	0.0000	0.0012	0.2324	0.0250	0.0515	26.8578
	periodic_ipf_100	0.0000	0.0180	0.2324	0.0250	0.1069	3944.3075
	periodic_ipf_1000	0.0000	0.0730	0.2324	0.0250	0.2692	2147.2080
	periodic_ipf_doubling	0.0000	0.0724	0.2324	0.0250	0.2662	2143.8918
	batch_c0.3	0.0009	–	0.2394	0.0247	–	1.6676
	stream_c0.3	0.0064	0.0162	0.2705	0.0380	0.0744	27.3696
	stream_c1	0.0056	0.0144	0.2544	0.0366	0.0700	27.7204
	sgd_all	0.0094	–	0.2006	0.0533	–	1607.2805
	mwu_all	0.0292	–	0.1613	0.1119	–	1695.7794
drift	raw	0.0055	0.0492	1.0000	0.0175	0.0739	0.0000
	ipf	0.0000	0.0000	0.9985	0.0136	0.0208	7.9924
	periodic_ipf_100	0.0000	0.0019	0.9985	0.0136	0.0211	213.8377
	periodic_ipf_1000	0.0000	0.0111	0.9985	0.0136	0.0285	109.3240
	periodic_ipf_doubling	0.0000	0.0192	0.9985	0.0136	0.0380	116.9140
	batch_c0.3	0.0000	–	0.9985	0.0136	–	1.1990
	stream_c0.3	0.0007	0.0021	0.7499	0.0141	0.0230	28.4732
	stream_c1	0.0005	0.0016	0.6812	0.0140	0.0242	28.7342
	sgd_all	0.0084	–	0.9150	0.0170	–	1152.7112
	mwu_all	0.0170	–	0.7980	0.0274	–	1512.5192
small	raw	0.0727	0.0782	1.0000	0.1026	0.1135	0.0000
	ipf	0.0000	0.0000	0.8859	0.0398	0.0682	19.5114
	periodic_ipf_100	0.0000	0.0169	0.8859	0.0398	0.0764	846.1529
	periodic_ipf_doubling	0.0000	0.0241	0.8859	0.0398	0.0803	831.3543
	batch_c0.3	0.0022	–	0.8922	0.0394	–	15.3335
	stream_c0.3	0.0041	0.0140	0.8188	0.0389	0.0705	59.0820
	stream_c1	0.0026	0.0093	0.7417	0.0410	0.0745	63.1449
	sgd_all	0.0096	–	0.8322	0.0393	–	675.3635
	mwu_all	0.0309	–	0.8711	0.0510	–	806.0989